



Παράλληλος και Κατανεμημένος Υπολογισμός I

Παράλληλος υπολογισμός (συν.)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Απεικόνιση και αναγωγή

Απεικόνιση και αναγωγή

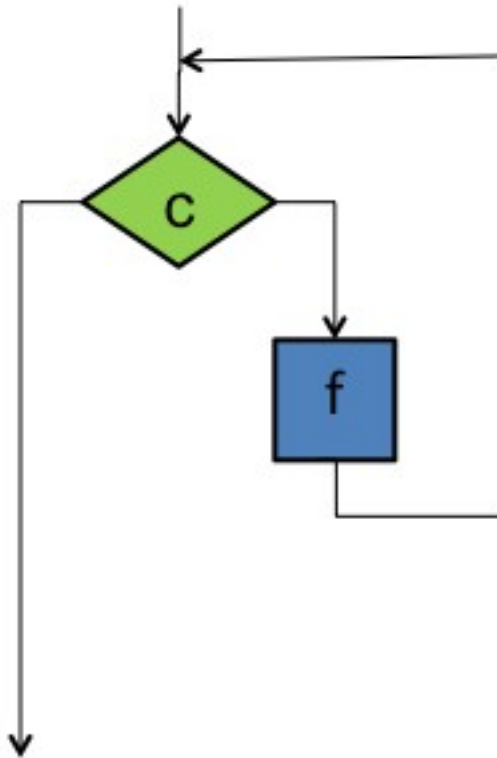
Η πιο συνηθισμένη αλγοριθμική κλάση, τόσο στο παραλληλισμό δεδομένων όσο και στον παραλληλισμό εργασιών, είναι η **απεικόνιση και αναγωγή (map, reduce)**.

- Απεικόνιση: μια δομή δεδομένων (ή μια απαριθμήσιμη επανάληψη) και σε κάθε στοιχείο (ή βήμα) εφαρμόζεται η ίδια συνάρτηση.
- Αναγωγή: συσσώρευση πολλών τιμών σε ένα αποτέλεσμα με βάση κάποιο προσεταιριστικό τελεστή.

Απλό παράδειγμα (απεικόνισης κ) αναγωγής: Άθροισμα ανύσματος

- Code/VecSum

Απαριθμήσιμη επανάληψη (for)



```
for (i = 0; i < n; ++i)
```

```
    y = f(x, i);
```

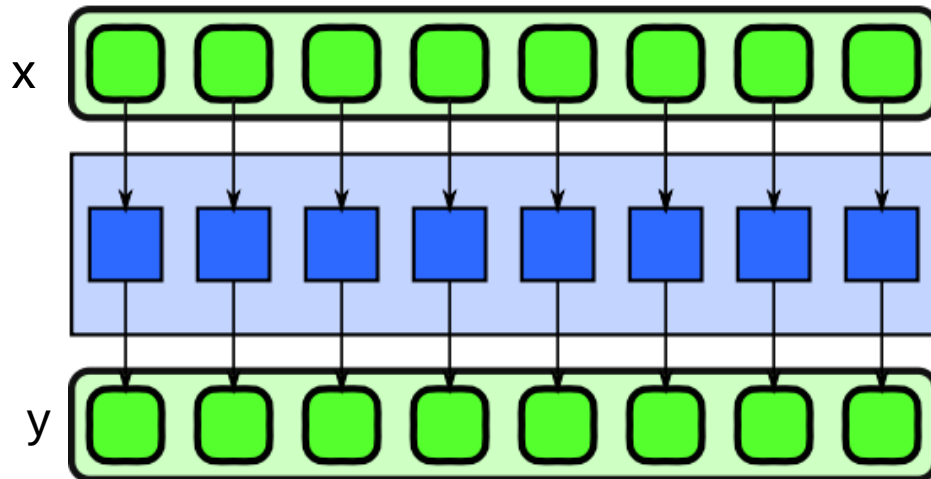
Σώμα της επανάληψης

- Η ακολουθιακή απαριθμήσιμη επανάληψη αντιστοιχεί σε αρκετά διαφορετικά πρότυπα παράλληλου σχεδιασμού.

Η διαφοροποίηση σχετίζεται με :

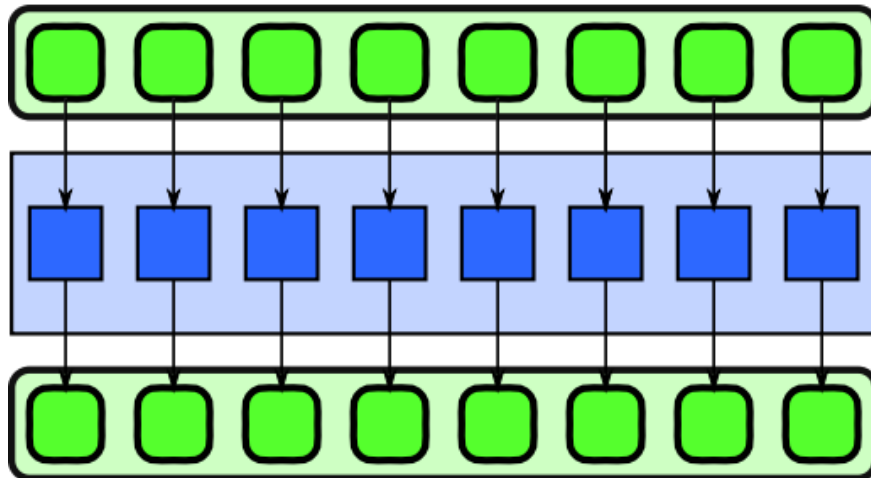
- Τις εξαρτήσεις μεταξύ των δεδομένων ή και βημάτων της επανάληψης
- Το απλούστερο /συνηθέστερο πρότυπο είναι η απεικόνιση

Απεικόνιση (map) 1



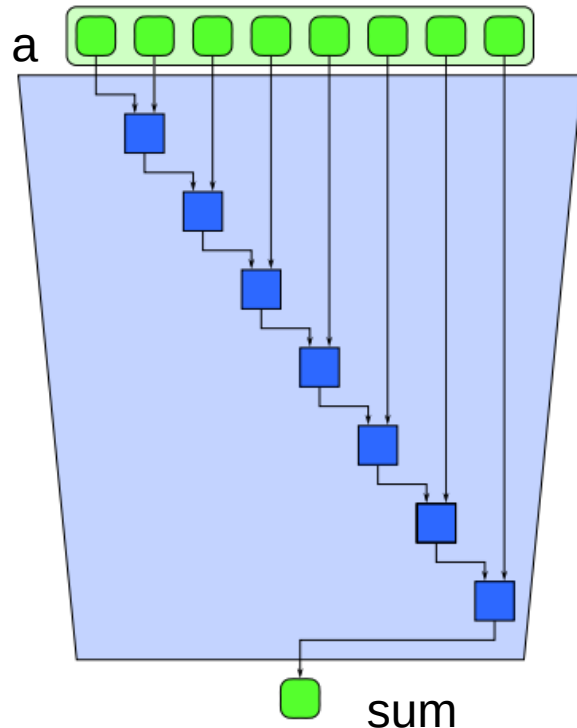
- Μπορεί να αναφέρεται σε: δομή δεδομένων (πχ string, αρχείο, άνυσμα, μητρώο, εικόνα, video, ήχος, βάση δεδομένων, δομές αντικειμένων) ή υπολογισμό συνάρτησης (πχ μαθηματικές συναρτήσεις, κρυπτογραφία, sat, what if σενάρια).

Απεικόνιση (map) 2



- Επανάληψη χωρίς εξαρτήσεις μεταξύ των βημάτων.
- Τυπική οργάνωση του παράλληλου προγράμματος ώστε να υλοποιηθεί η απεικόνιση: Single Program Multiple Data (SPMD).
- Κάθε παράλληλη εργασία (πχ νήμα ή διεργασία) υπολογίζει συνήθως μια ομάδα στοιχείων.

Αναγωγή (reduce) 1



```
sum = 0;  
for (i=0; i<N; ++i)  
    sum += a[i]
```

- Συσσώρευση των στοιχείων ενός ανύσματος με βάση έναν **προσεταιριστικό** τελεστή, πχ πρόσθεση, max.
- Συνήθως προηγείται επεξεργασία τύπου απεικόνισης και κατάληξη με κάποιου τύπου αναγωγή.

Υλοποίηση αναγωγής 1

- Υπολογισμός μερικού αθροίσματος από κάθε παράλληλη εργασία (κατανομή όπως στην απεικόνιση).
- Εναλλακτικές λύσεις για την αναγωγή:
 - Άθροιση από τη main με for loop.
 - (α) Κάθε παράλληλη εργασία αποθηκεύει το μερικό άθροισμα σε ξεχωριστή θέση ενός ανύσματος.
 - (β) `Επιστροφή` μερικών αθροισμάτων και άθροιση κατά τον τερματισμό των νημάτων.



```
Shared sums[P] = {0,0, ...,0}
```

```
Main {
```

```
    for (i=0; i<P; i++)  
        create/start Task (i);
```

```
    sum = 0;
```

```
    for ((i=0; i<P; i++) {  
        wait/join Task (i);  
        sum += sums[i];
```

```
    }
```

```
}
```

```
Task(i) {
```

```
    block = N / P;
```

```
    start = i * block;
```

```
    stop = start + block;
```

```
    if (i == P-1) stop = N;
```

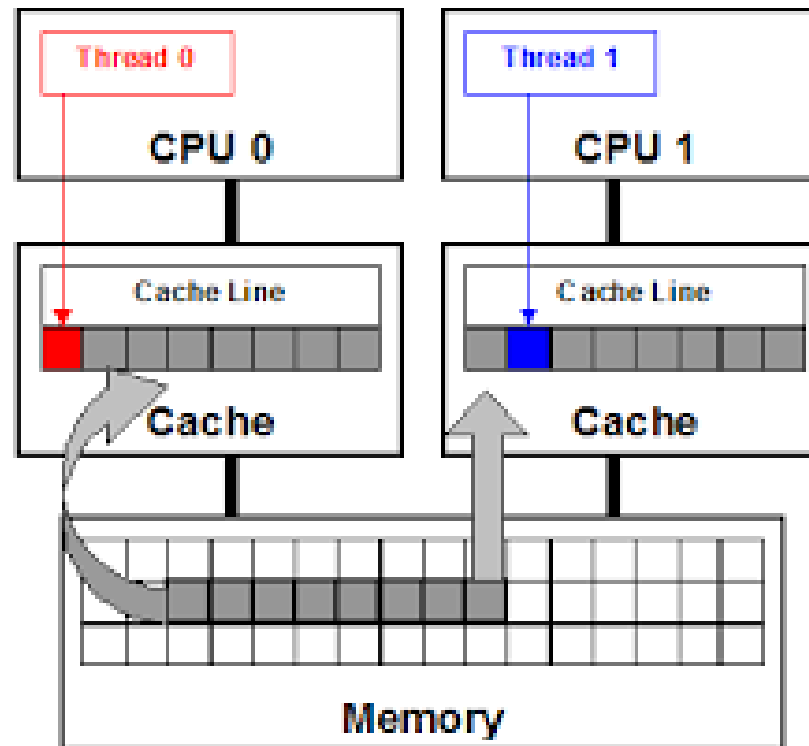
```
    for (j = start; j<stop; j++)
```

```
        sums[i] += a[j];           //Multiple array references
```

```
}
```

Αναγωγή με
μοιραζόμενο πίνακα
(λάθος)

Ψευδής διαμοιρασμός False Sharing



- Ο μοιραζόμενος πίνακας `sums[ ]` βρίσκεται σε όλες τις κρυφές μνήμες των επεξεργαστών. Ο κάθε επεξεργαστής τροποποιεί ένα στοιχείο του πίνακα.
- Ο αλγόριθμος συνοχής κρυφής μνήμης εξετάζει γραμμές της κρυφής μνήμης, όχι στοιχεία.
- Ευτυχώς οι μεταγλωττιστές συνήθως το διορθώνουν εισάγοντας τα απαιτούμενα κενά στοιχεία ενδιάμεσα (padding).



```
Shared sums[P] = {0,0, ...,0}
Main {
```

```
    for (i=0; i<P; i++)
        create/start Task (i);
    sum = 0;
    for (i=0; i<P; i++) {
        wait/join Task (i);
        sum += sums[i];
    }
}
```

```
Task(i) {
    block = N / P;
    start = i * block;
    stop = start + block;
    if (i == P-1) stop = N;
    local_sum = 0; //Local variable
    for (j = start; j<stop; j++)
        local_sum += a[j];
    sums[i] = local_sum; //Single array reference
}
```

Αναγωγή με
μοιραζόμενο πίνακα
(σωστό: Τοπικότητα)

Αναγωγή με τοπικότητα και επιστροφή τιμής

```
Main {  
    for (i=0; i<P; i++)  
        create/start Task (i);  
    sum = 0;  
    for (i=0; i<P; i++) {  
        wait/join Task (i);  
        sum += get(Task[i].local_sum); // Task[i].get();  
    }  
}  
Task(i) {  
    local_sum = 0; // Main can access local_sum  
                  // e.g. instance variable  
  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
    for (j = start; j<stop; j++)  
        local_sum += a[j];  
  
    // Some libraries have return local_sum  
}
```

Υλοποίηση αναγωγής 2

Άθροιση σε μοιραζόμενη μεταβλητή

Κάθε νήμα προσθέτει το μερικό του άθροισμα.
Απαιτείται αμοιβαίος αποκλεισμός (μοιραζόμενος μετρητής).

- Προσοχή στη σωστή χρήση των τοπικών μεταβλητών και του μηχανισμού ταυτοχρονισμού.
- Η λανθασμένη εφαρμογή μπορεί να οδηγήσει σε υπερβολικές καθυστερήσεις.

```
Shared lock_type l;  
    sum = 0;
```

```
Main {  
    for (i=0; i<P; i++)  
        create/start Task (i);  
    for (i=0; i<P; i++) {  
        wait/join Task (i);  
    }  
}
```

```
Task(i) {  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
    for (j = start; j<stop; j++) {  
        lock(l);  
        sum += a[j];  
        unlock(l);  
    }  
}
```

//Multiple lock/unlock

Αναγωγή με
μοιραζόμενη
μεταβλητή και
κλείδωμα (λάθος)



```
Shared lock_type l;  
    sum = 0;
```

```
Main {  
    for (i=0; i<P; i++)  
        create/start Task (i);  
    for (i=0; i<P; i++) {  
        wait/join Task (i);  
    }  
}
```

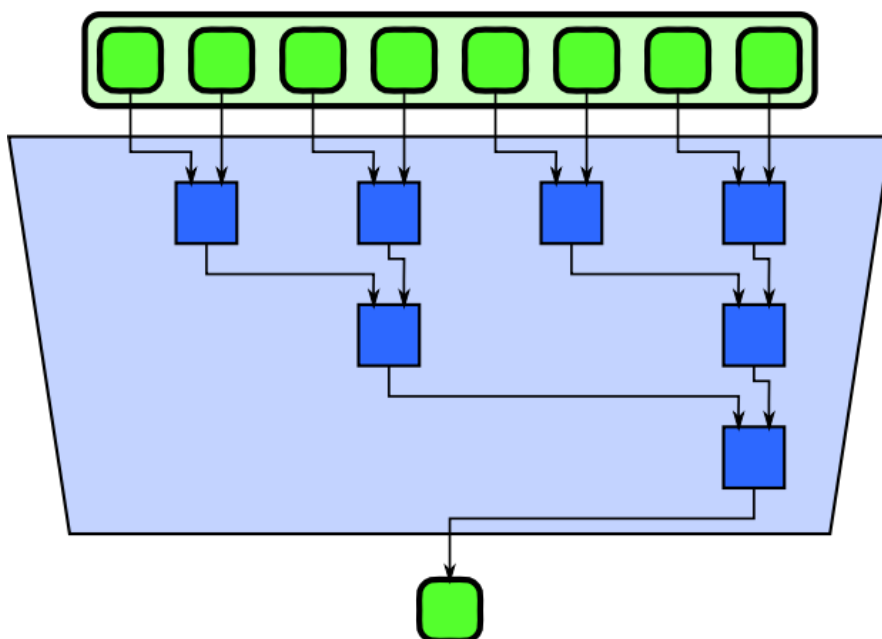
```
Task(i) {  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
    local_sum = 0; // local variable  
    for (j = start; j<stop; j++)  
        local_sum += a[j];  
    lock(l);  
    sum += local_sum; //One lock/unlock on exit  
    unlock(l);  
}
```

Αναγωγή με
μοιραζόμενη
μεταβλητή και
κλείδωμα (σωστό:
Τοπικότητα)

Υλοποίηση αναγωγής 3

Δενδρική δομή

- Τα νήματα προσθέτουν τα μερικά αθροίσματά τους ανά δύο, μετά ανά τέσσερα κ.ο.κ. Λογαριθμικός χρόνος αλλά σύνθετη υλοποίηση ή αναδρομή.



Αρίθμηση εργασιών: 8 εργασίες

111 110 101 100 011 010 001 000

111 110 101 100 011 010 001 000

111 110 101 100 011 010 001 000

111 110 101 100 011 010 001 000

Ολοκλήρωση σε $\log_2$ βήματα
 $\log_2 8 = 3$ βήματα

Shared sums[P] = {0,0, ...,0}

```
Main {  
    for (i=0; i<P; i++)  
        create/start Task (i);  
    for (i=0; i<P; i++)  
        wait/join Task (i);  
    sum = sums[0];  
  
}  
Task(i) {  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
    local_sum = 0;  
    for (j = start; j<stop; j++)  
        local_sum += a[j];  
    sums[i] = local_sum;  
    reduction_tree(i);  
}
```

Αναγωγή με
δυναμικό δένδρο 1

Αναγωγή με δυαδικό δένδρο 2

```
reduction_tree (i) {  
    for (j=1; j<=log2P; j++)  
        barrier();    // all threads must synchhronize here*  
        if ((i % 2^j) == 0)  
            sums[i] += sums[i+2^(j-1)];  
}
```

Παράδειγμα για $P = 8$

j=1 sums[0] += sums[1], sums[2] += sums[3]
 sums[4] += sums[5], sums[6] += sums[7]

j=2 sums[0] += sums[2], sums[4] += sums[6]

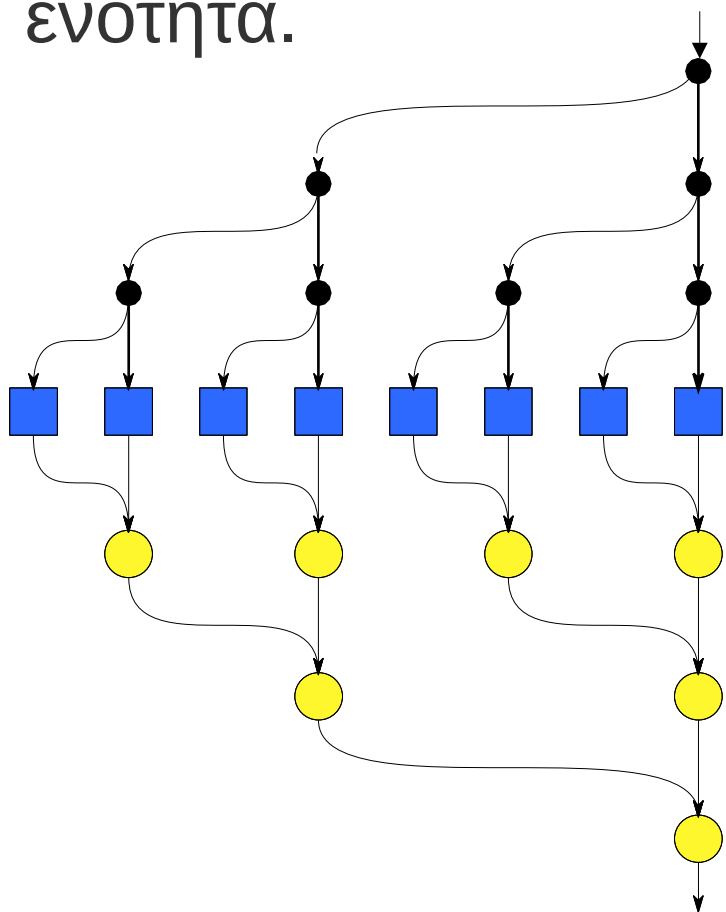
j=3 sums[0] += sums[4]

*Συζητούμε τα φράγματα παρακάτω

Υλοποίηση αναγωγής 4

Αναδρομικός παραλληλισμός (divide and conquer)

- Αναδρομική υλοποίηση της δενδρικής δομής για την αναγωγή . Θα τον συζητήσουμε σε επόμενη ενότητα.



Ολοκλήρωση σε $\log_2$ βήματα
 $\log_2 8 = 3$ βήματα

$$\text{Sum}(0,7) = \text{Sum}(0,3) + \text{Sum}(4,7)$$

$$\text{Sum}(0,3) = \text{Sum}(0,1) + \text{Sum}(2,3)$$

$$\text{Sum}(4,7) = \text{Sum}(4,5) + \text{Sum}(6,7)$$

..

Σύνοψη απεικόνισης αναγωγής 1

- Έστω ένα πρόβλημα μεγέθους N που επιλύεται με τη μέθοδο απεικόνισης αναγωγής. Έστω ότι έχουμε στη διάθεσή μας P επεξεργαστές.
- Βήματα που απαιτούνται για την απεικόνιση: N/P
- Βήματα που απαιτούνται για την αναγωγή:
με τους πρώτους 3 τρόπους: P
με τους 2 τελευταίους τρόπους: $\log_2 P$
- Συνολικά για την παράλληλη επίλυση του προβλήματος με απεικόνιση και αναγωγή απαιτούνται:
 $N/P + P$ ή $N/P + \log_2 P$ βήματα

Σύνοψη απεικόνισης αναγωγής 2

- Αν το P είναι μικρό τότε η διαφορά P και $\log_2 P$ είναι αμελητέα (το N/P κυριαρχεί)
πχ αν έχουμε $P=16$ επεξεργαστές
 $P = 16$ και $\log_2 P = 4$
- Αν το P είναι αρκετά μεγάλο τότε ο όρος $\log_2 P$ μπορεί να είναι σημαντικός
πχ αν $P=1024$ επεξεργαστές
 $P = 1024$ και $\log_2 P = 10$



Ένθετες επαναλήψεις με αναγωγή

Ένθετες επαναλήψεις με αναγωγή

- Πολύ συνηθισμένη τεχνική στην επεξεργασία διανυσμάτων, δισδιάστατων και πολυδιάστατων μητρώων.

Ενδεικτικές εφαρμογές:

- Επιστημονικοί υπολογισμοί
- Βιοπληροφορική
- Γραφικά υπολογιστών
- Επεξεργασία εικόνας και video
- Μηχανική μάθηση, τεχνητά νευρωνικά δίκτυα
- Δεδομένα μεγάλου όγκου

Παράδειγμα: πολλαπλασιασμός μητρώων

- Εμβληματική εφαρμογή, βασικό μετροπρόγραμμα επιδόσεων παράλληλων υπολογισμών.
- Απλός τρόπος αντιμετώπισης: παραλληλοποίηση μόνο του εξωτερικού βρόχου: Η εσωτερική επανάληψη εκτελεί (ακολουθιακές) αναγωγές.
- Πρόβλημα: πολλαπλές προσπελάσεις στη μνήμη μπορεί να μειώσουν την αριθμητική πυκνότητα
- Υπάρχουν πιο αποδοτικές μέθοδοι, που εκμεταλλεύονται πλήρως την ιεραρχία της μνήμης, και εφαρμόζουν πολλαπλές τεχνικές παραλληλισμού.
- Ειδικές βιβλιοθήκες (πχ <http://www.openmathlib.org/OpenBLAS/>).

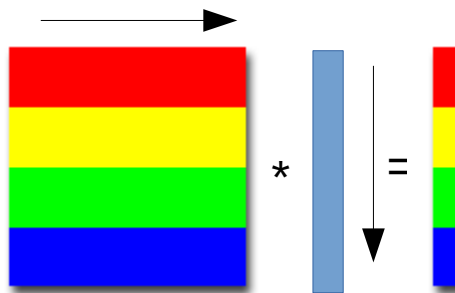
Πολλαπλασιασμοί μητρώων, ανυσμάτων 1

Πολλαπλασιασμός μητρώου ανύσματος:

```
for (i = 0; i < n; i++) {  
    c[i] = 0  
    for (j = 0; j < n; j++)  
        c[i] = c[i] + a[i,j] * b[j]
```

```
}
```

task(i)

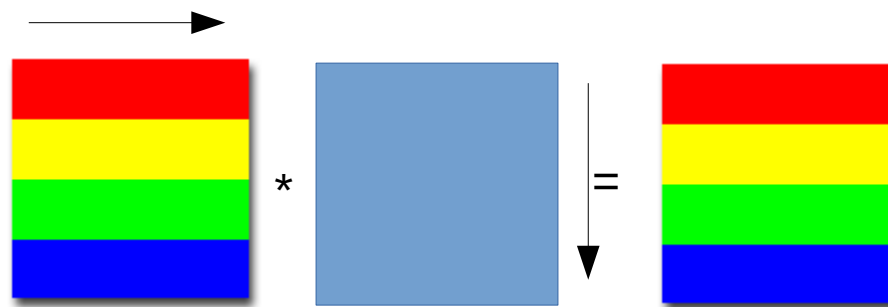


Πολλαπλασιασμός μητρώων:

```
for (i = 0; i < n; i++) {  
    c[i,j] = 0  
    for (j = 0; j < n; j++)  
        for (k=0; k<n; k++)  
            c[i,j] = c[i,j] + a[i,k] * b[k,j]
```

```
}
```

task(i)



Απλή ταύτιση προτύπων (συμβολοσειρών) 1

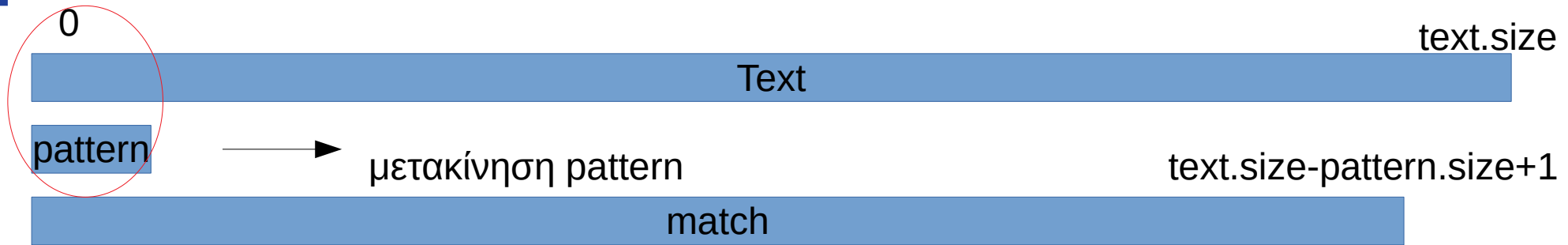
- Μερικές εφαρμογές:

Text search, Spell Check, Autocomplete
Plagiarism Detection
Bioinformatics,
Digital Forensics
Spam Filters
Intrusion Detection
Search Engines

- Μερικοί αλγόριθμοι

<https://www-igm.univ-mlv.fr/~lecroq/string/>

Απλή ταύτιση προτύπων (συμβολοσειρών) 2



```
for (j = 0; j <= text.size - pattern.size; j++) {
```

```
    for (i = 0; i < pattern.size; i++)  
        if (pattern[i] != text[i + j]) break
```

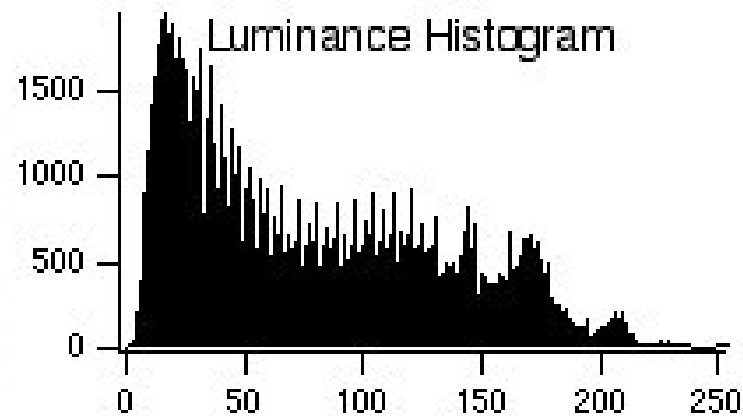
Σύγκριση, αναγωγή

```
        if (i == m)  
            match[j] = 1  
        else  
            match[j] = 0
```

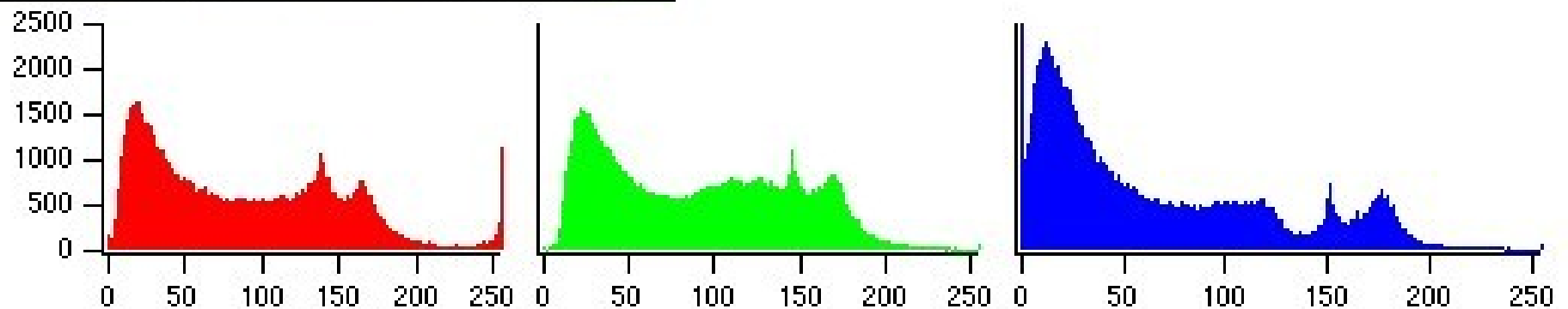
```
}
```

Αναγωγή συγκρίσεων στο πίνακα match.

Ιστόγραμμα εικόνας 1



$$\text{Luminance} = (r + g + b) / 3$$



Μερικές εφαρμογές: <https://www.baeldung.com/cs/image-histograms>

Ιστόγραμμα εικόνας 2

<https://rosettacode.org/wiki/Bitmap/Histogram#Java>

```
for (i = 0; i < 256; i++) {  
    r_hist[i] = 0; g_hist[i] = 0; b_hist[i] = 0;  
    luminance_hist[i] = 0;  
}
```

```
for (i = 0; i < ImageRows; i++)  
    for (j = 0; j < ImageColumns; j++) {  
        r = get.r(pixel[i,j]); g = get.g(pixel[i,j]); b = get.b(pixel[i,j]);  
        r_hist[r]++; g_hist[g]++; b_hist[b]++;  
  
        lunminance = (r + g + b) / 3;  
        luminance_hist[lunminance]++;  
    }
```

Αναγωγή σε πίνακες r_hist, g_hst, b_hist, luminance_hist.



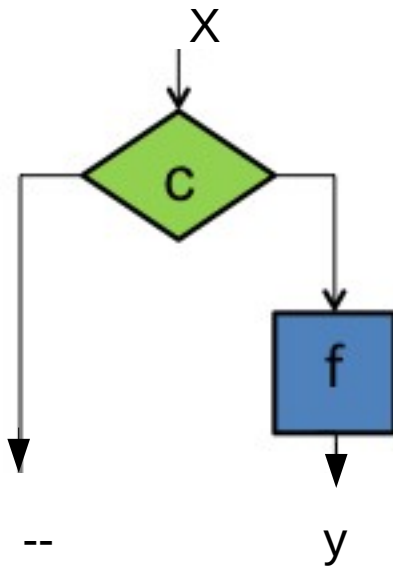
Απεικόνιση- Επιλογή - Αναγωγή



Απλά παραδείγματα

- Code/MapReduce

Απλή επιλογή (φιλτράρισμα) (if then) 1



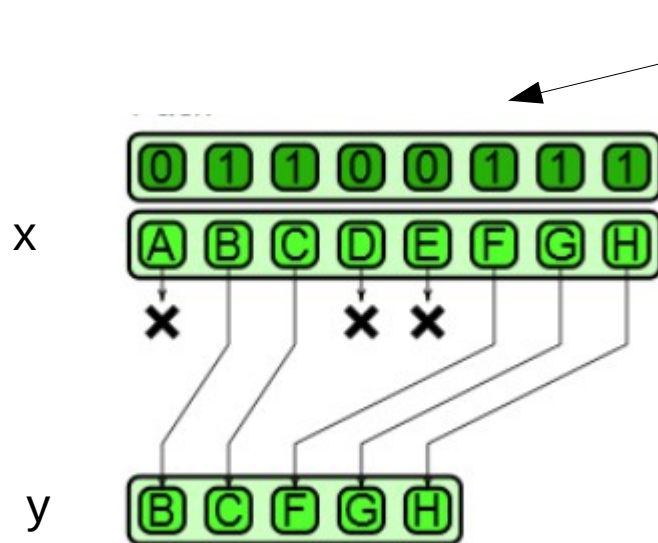
if (condition(x)) y = f(x)

- Η επιλογή μπορεί να τεθεί μέσα σε επανάληψη ώστε τα x, y να είναι ανύσματα, μητρώα και γενικά λίστες τιμών

```
count = 0;  
for (i = 0; i < n; ++i)  
    if (condition(x[i]))  
        y[count++] = x[i]
```

Επιλογή σε μια δομή
(άνυσμα, 1δ πίνακας):
Παραγωγή μιας νέας δομής
με Length.y <= Length.x

Απλή επιλογή (φιλτράρισμα) (if then) 2



- Η επιλογή μπορεί να τεθεί μέσα σε επανάληψη ώστε τα x , y να είναι ανύσματα, μητρώα και γενικά λίστες τιμών

Μπορεί να απαιτείται δημιουργία νέας δομής.
Όχι απαραίτητα η ίδια σειρά στοιχείων στις δύο δομές.
Αν απαιτείται ίδια σειρά ο αλγόριθμος λέγεται Pack,
και είναι πιο σύνθετος.

```
Shared lock_type l;  
    count = 0;  
    a = { ... }, y = { ...};
```

```
Main {  
    for (i=0; i<P; i++)  
        create/start Task (i);  
    for (i=0; i<P; i++) {  
        wait/join Task (i);  
    }  
}  
Task(i) {  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
    for (j = start; j<stop; j++)  
        if (condition(a[j])){  
            lock(l);  
            y[count++] = a[j];  
            unlock(l);  
        }  
}  
}
```

Απλή επιλογή
(φιλτράρισμα)
(if then) 3

MapReduce: απεικόνιση (φιλτράρισμα) αναγωγή

- Μεγάλο μέρος των τυπικών λειτουργιών επεξεργασίας μηχανών αναζήτησης, συστημάτων συστάσεων και γενικά δεδομένων μεγάλου όγκου ακολουθεί το μοντέλο MapReduce.
- Μεταξύ της απεικόνισης και της αναγωγής μπορεί να μεσολαβεί και `filter()`, δηλαδή παραγωγή ανύσματος `z` από το `y` με βάση κάποιο κριτήριο:

```
sum = 0
for (i = 0; i < n; i++)
    if (condition(f(x[i])))
        sum = sum + f(x[i])
```

Μια πιθανή ακολουθιακή μορφή
Δεν απαιτείται νέα δομή
δεδομένων.

Σύνοψη απεικόνισης- επιλογής - αναγωγής

1. Το main δημιουργεί εργασίες,
2. Το main κατανέμει τα δεδομένα, συνήθως με χρήση στατικής κατανομής,
3. Όλες οι εργασίες εκτελούν τον ίδιο κώδικα (τοπικά for loops) αλλά σε διαφορετικά δεδομένα, με μέγιστη δυνατή χρήση της τοπικότητας των υπολογισμών, ώστε τα μερικά αποτελέσματα συσσωρεύονται τοπικά.
4. Το main συγκεντρώνει τα αποτελέσματα. Η τελική συγκέντρωση των τοπικών αποτελεσμάτων γίνεται είτε από το main είτε από τις εργασίες αλλά πάντα στο τελικό στάδιο των υπολογισμών, όχι μέσα στα τοπικά for loops.
5. Το main αναμένει τις εργασίες να ολοκληρωθούν.

Απλά παραδείγματα απεικόνισης, φιλτραρίσματος και αναγωγής:

- Άθροισμα θετικών / αρνητικών αριθμών σε ένα πίνακα ακεραίων αριθμών
- Εύρεση μεγίστου / ελαχίστου σε ένα πίνακα ακεραίων αριθμών

Παρόμοια με Code/VecSum

- Εύρεση συχνότητας εμφάνισης χαρακτήρων σε ένα αρχείο
 - Εύρεση συχνότητα εμφάνισης λέξεων σε ένα αρχείο
- Code/MapReduce

Ταυτόχρονες δομές δεδομένων

- Τα ενδιάμεσα και τα τελικά αποτελέσματα των παράλληλων `map()` `filter()` `reduce()` μπορεί να αποθηκεύονται σε διάφορες δομές δεδομένων, όχι απαραίτητα πίνακες (arrays).
- Συχνά ο χειρισμός των ενδιάμεσων δεδομένων --πχ μετά το `filter()`-- περιλαμβάνει και κάποιου είδους αναδιάταξη (shuffle) ή ταξινόμηση (sort), ώστε, για παράδειγμα να πετύχουμε εξισορρόπηση φορτίου στο `reduce()`.
- Συνδυασμός παράλληλου `map()` `filter()` `reduce()` με ταυτόχρονες δομές δεδομένων.

Παραδείγματα 1

- Έστω μια συλλογή από αρχεία κάποιου κειμένου, (HTML, βιβλία, εικόνες, video) ή απλά μια μεγάλη συλλογή από μεγάλες συμβολοσειρές (ακολουθίες DNA, πρωτεϊνών). Θέλουμε να βρούμε:
- Συχνότητες εμφάνισης μιας συμβολοσειράς s στα στοιχεία της συλλογής (διάφορες υποπεριπτώσεις) ή
- Συχνότητες εμφάνισης της s στα στοιχεία της συλλογής πάνω από μια προκαθορισμένη συχνότητα f .
- Τις πιο συχνές κοινές ακολουθίες (λέξεων, αμινοξέων, πρωτεϊνών κλπ).

Παραδείγματα 2

- Έστω ένας πίνακας (μητρώο) ή βάση δεδομένων που περιέχει χρήστες (με κάποια χαρακτηριστικά τους) και πχ αγοραστικές προτιμήσεις (ή επισκέψεις σε σελίδες, ή μηνύματα σε επαφές ή posts ..). Θέλουμε να βρούμε
- Τον μέσο όρο αγορών συγκεκριμένου τύπου προϊόντος από χρήστες μιας κατηγορίας ή
- Τον μέσο όρο posts από χρήστες ενός ηλικιακού εύρους ή φύλου ή τοποθεσίας ή...
- Μια ομάδα χρηστών που έχει κοινές επισκέψεις σε συγκεκριμένες σελίδες..
- Αριθμό συνδέσεων, συχνές ακολουθίες ενεργειών χρηστών..

Παραδείγματα 3

- Έστω μια συλλογή από ιστοσελίδες, χρήστες κοινωνικών δικτύων. Θέλουμε να βρούμε τις σελίδες ή τους χρήστες με περισσότερες από f εισερχόμενες συνδέσεις ή εξερχόμενες συνδέσεις ή συνδυασμό των παραπάνω.
- Συστήματα συστάσεων: ομαδοποίηση χρηστών με βάση τις προτιμήσεις τους σε προϊόντα, ιστοσελίδες. Ομαδοποίηση προϊόντων, ιστοσελίδων με βάση τη συνάφεια: ο χρήστης X μετά το προϊόν A είδε το προϊόν B . Συστάσεις: Αν ο χρήστης Y ανήκει στην ίδια ομάδα με τον X και έχει δέη το προϊόν A αλλά όχι το B τότε πρότεινε του να δει το B .

MapReduce Frameworks

- Προσαρμογή αλγορίθμων στο πρότυπο απεικόνιση (φιλτράρισμα) αναγωγή
- Ανάπτυξη ειδικών frameworks για τη σχετικά εύκολη ανάπτυξη σχετικών εφαρμογών πχ
- Apache Hadoop, Apache Spark
- Σε πολλές γλώσσες / βιβλιοθήκες υπάρχουν έτοιμες λειτουργίες `map()`, `reduce()`, `parallel()`
- πχ Java Streams (δες σχετικό κώδικα)